

大作业报告

暂时不能给你明确的答复队（组长：陈勇整；组员：袁方涵，胡翔宇）

一、程序功能介绍

本项目是基于 QT 以 python 语言开发的单机回合制战术对战游戏——“暗夜竞技场”。游戏以双角色棋盘对战为核心玩法，将角色选择、技能搭配、回合制行动、位置判定、状态结算、AI 对战、账号记录和胜负展示等功能整合到一个完整的桌面应用中。玩家通过主页图形界面完成战前配置，并在有限大小的棋盘中与 AI 控制的敌方角色进行对战。游戏的主要玩法是通过对不同的角色的技能与数值的理解和弥补性 basicskills 的选择来针对性补强对应的角色的强势和弱弱点，在战斗过程中，对手的下一步都是不可见的，而我方使用各种技能都有对应的能量消耗和相应技能进入 cd，在本游戏的设计中，很少有位移技能和恢复技能，因此如果没有合适的走位和预判，很容易耗尽能量进入 cd 进入被动挨打的阶段。因此，本游戏考研玩家的策略与预判。

用户启动程序后，首先就会进入登录界面，可以进行账号注册和登录。在注册过程中会要玩家选择名字和图像，密码，之后是登录成功后进入主界面，也就是战前准备大厅。大厅中会展示玩家账号信息、头像、胜负统计、当前选择的角色和通用技能配置。玩家可以从大厅进入角色选择界面、通用技能选择界面、AI 对手选择界面，也可以直接开始战斗。这样的流程使程序具备较完整的对战系统。

角色选择是本游戏的重要玩法和特点之一。游戏内设计了多名具有不同战斗定位和专属技能的角色，从大类上分为三大类角色，分别是以魔力强血量和防御低的法师大类，包括火焰法师、大地法师、藤蔓法师、空间法师，以均衡和强化为特点的战士类，包括战争领主、铁链角斗士、幽影刺客、血誓狂战，最后还有各种各样特色技能的兽人类，如狂翼鸟人、幻鳞蜥客、殉爆骨将和乱魂巫医等。不同角色拥有不同的生命值、攻击力、能量上限、移动能力和对应的专属技能。角色定位覆盖爆发输出、控制、机动、持续伤害、防御反击、范围压制等多种类型，玩家在战前选择阶段就需要考虑自己喜欢的战斗风格和技能组合思路进行游戏的初始配置。

每个角色除基础数值外，还拥有两个具有对应特点的专属技能。例如火焰法师擅长通过火焰范围技能造成高额伤害；铁链角斗士可以通过断罪锁链建立牵制关系，并在后续回合根据距离结算伤害；幽影刺客通过毒影飞镖和蚀骨催毒制造持续毒伤与爆发收割；乱魂巫医可以施加随机移动和能量损耗效果，干扰敌方行动；空间法师则通过无敌、换位和反弹机制进行反制。诸如此类，不同角色的技能机制不是简单的数值差异，而是体现为不同的行动机制、目标选择方式和状态效果。下图为相关角色一览表：

角色技能特点表（一）：法师与控制角色			
角色	定位	技能特点	专属技能介绍
火焰法师	法师 / 爆发输出	以高伤害范围攻击为核心，技能规则直接，适合通过预判敌方位置打出爆发。	火龙卷：选择中间行非边缘格，对目标周围 3×3 范围造成 30 点范围伤害。 烈焰横扫：攻击自己所在整行，敌人在该行时受到 30 点伤害。
大地法师	法师 / 控场	通过泥潭、石柱和无视防御伤害限制敌方走位，偏向区域控制。	泥潭迸发：首次放置泥潭；场上已有泥潭时引爆十字范围，造成 20 点无视防御范围伤害。 岩突震落：选择同行或同列方向格，命中方向上的敌人时造成 20 点无视防御伤害。
藤蔓法师	控制 / 吸能型法师	通过区域吸能和技能禁锢限制敌方行动能力，适合持续控制和资源压制。	森罗汲取：选择 2×2 区域，敌人在区域内时造成 20 点伤害，并吸取敌方能量。 万藤封禁：选择目标格，命中时阻止敌方本轮技能生效，并限制后续位移。
空间法师	法师 / 反制机动	强调防御反制和位置交换，能够利用敌方攻击行为制造反弹效果。	星界守护：本轮遭受，下一轮获得无敌状态，用于规避关键伤害。 维度换位：选择目标格，命中时与敌人交换位置，并使敌方本轮攻击反弹给自身。

角色技能特点表（二）：战士角色			
角色	定位	技能特点	专属技能介绍
战争领主	战士 / 压阵强化	通过战令逐步强化自身，再利用战意机制压制敌方移动或触发追击。	战令三式：一式提供位移强化和首次反击免疫；二式提供攻击伤害 +5、范围 +1。王威压阵：获得战意；战意不足时削弱敌方移动，战意达到 2 点可相邻追击 15 点伤害。
	战士 / 牵制近战	依靠锁链建立牵制关系，根据双方距离或相邻关系结算伤害。	铁链锁链：首次 3 格内牵制；牵制后再次选择 3 格内目标格，命中造成 5 + 5 × 当前距离伤害。铁链脱离：判定自身上下左右相邻格，无锁链 14 点，有锁链 20 点并缩短锁链。削弱敌方攻击。
幽影刺客	战士 / 毒伤持续	通过投掷技毒，再利用催毒扩大持续伤害收益，适合消耗和收割。	毒影飞镖：选择同行或同列方向格，命中直线敌人造成 10 点伤害并中毒 3 回合。铁毒催毒：命中已中毒敌人时追加一次本轮毒伤，通常 5 点。
	战士 / 低血爆发	血量越低爆发越强，通过血疯、血怒和不死状态换取高风险输出。	裂创斩：持续 2 格内攻击，造成 12 + 4 × 血疯指数伤害；血怒时追加 5 点。血誓不灭：生命不高于 35% 时可用，获得不死和血怒，并转化为横刃形态。

角色技能特点表（三）：兽人与特殊角色			
角色	定位	技能特点	专属技能介绍
狂翼鸟人	兽人 / 飞袭印记	拥有强机动能力，可以通过任意格飞行和列印记制造持续压制。	冲天急袭：飞往任意空格，落点距离敌人不超过 1 格时造成 18 点伤害。坠羽印记：选择一列留下 3 回合印记；前两次造成 8 点伤害，最后爆炸造成 20 点伤害。
幻鳞刺客	兽人 / 隐身偷袭	通过隐身强化突袭技能，具备拉近敌人和高伤害偷袭能力。	幽影变色：进入隐身 2 回合，强化断舌奇袭。断舌偷袭：选择同线 3 格内目标格，命中时拉近敌人并造成伤害；普通 20 点，隐身 28 点。
魂爆骨将	兽人 / 自毁复生	以高风险换血和复活机制为核心，适合在关键局面制造强制交换。	同归于尽：若敌人在 3 × 7 范围内，双方各受 30 点无视防御伤害；敌人离开范围则自己死亡。残骨重组：进入残骨状态；首次死亡时复活并恢复 35 点生命。
乱魂巫医	兽人 / 随机诅咒	通过随机移动、能量扣除和生命反噬打乱敌方节奏，偏控制和干扰。	魔咒乱魂：使敌方从下一回合开始随机移动 2 回合，每回合扣除 8 点能量。失控反噬：选择 2 × 2 区域，命中乱魂中的敌人时使当前生命减半，否则增加随机移动。

通用技能系统是本项目的另一项核心功能。玩家可以在战前从多个通用技能中选择搭配，包括普通攻击、重斩、贯穿射击、范围爆破、翻滚、钩步、盾墙、反击姿态、能量充盈、沉默、小治疗等。通用技能覆盖攻击、位移、防御、辅助和控制等类别。甚至有很多通用技能不逊色于角色的技能效果，技能选择界面会展示每个技能的功能说明、冷却时间和能量消耗，帮助玩家理解技能效果并做出配置决策。通用技能与角色专属技能共同构成了玩家的实际战斗牌组，使同一角色也可以通过不同通用技能配置形成不同打法。再次增加游戏的可玩性和灵活性。下面是通用技能的相关介绍表：

通用技能介绍表（一）：基础行动与生存			
技能	类型	冷却 / 消耗	范围 / 目标
技能介绍			
普通攻击	攻击	CD 0 / 消耗 0	目标格：1 格
技能描述：基础攻击动作，造成物理伤害。敌人生命值降低，造成物理伤害。技能效果：基础伤害 10 点，暴击伤害 150%。			
位移	位移	CD 2 / 消耗 15	目标格：1 格
技能描述：快速移动到指定格子，无视障碍。技能效果：位移距离 1 格，冷却时间 2 秒。			
冲刺	位移	CD 2 / 消耗 20	目标格：1 格
技能描述：快速移动到指定格子，无视障碍。技能效果：位移距离 1 格，冷却时间 2 秒。			
防御	防御	CD 3 / 消耗 20	目标格：自身
技能描述：进入防御姿态，降低受到的伤害。技能效果：防御姿态持续 2 回合，受到伤害减少 50%。			
治疗	治疗	CD 4 / 消耗 10	无范围
技能描述：恢复自身生命值。技能效果：恢复生命值 10 点，冷却时间 4 秒。			

通用技能介绍表（二）：攻击与机动			
技能	类型	冷却 / 消耗	范围 / 目标
技能介绍			
小范围	辅助	CD 7 / 消耗 40	无范围
技能描述：对小范围敌人造成伤害。技能效果：对小范围敌人造成伤害，冷却时间 7 秒。			
重斩	攻击	CD 2 / 消耗 25	目标格：1 格
技能描述：对指定格子造成重击伤害。技能效果：对指定格子造成重击伤害，冷却时间 2 秒。			
贯穿射击	攻击	CD 3 / 消耗 20	同行 / 同列 / 同线
技能描述：对指定格子造成贯穿伤害。技能效果：对指定格子造成贯穿伤害，冷却时间 3 秒。			
范围爆破	攻击	CD 3 / 消耗 40	中心格 < 2 格
技能描述：对指定格子造成范围伤害。技能效果：对指定格子造成范围伤害，冷却时间 3 秒。			
翻滚	位移	CD 3 / 消耗 20	目标格：1 格
技能描述：对指定格子造成翻滚伤害。技能效果：对指定格子造成翻滚伤害，冷却时间 3 秒。			

通用技能介绍表（三）：控制、防御与冷却管理				
技能	类型	冷却 / 能耗	范围 / 目标	效果介绍
预判	控制	CD 3 / 能耗 20	目标格：3格	预判敌方技能释放，提前对敌人释放人形激光攻击判定，造成中等范围伤害并强制位移。
屏障	防御	CD 5 / 能耗 30	无范围格	当前回合受到伤害减少 10%，造成物理伤害时生效，造成非物理伤害时无效。
反击姿态	控制	CD 4 / 能耗 20	无范围格	成功受到伤害时强制进入反击姿态，自己伤害 20% 恢复，造成物理伤害无效。
能量耗尽	控制	CD 4 / 能耗 30	无范围格	当前回合受到伤害强制减少 1 格，命中时强制进入昏迷，持续回合数一定数量。
沉默	控制	CD 4 / 能耗 15	目标格：3格	沉默 1 回合的技能，敌人无法释放技能且敌人无法行动，造成物理伤害无效。

战斗部分采用回合制同步结算机制。每个回合中，玩家先选择本回合使用的技能和目标格，AI 也会根据当前局势选择行动。之后系统统一结算双方行动，而不是简单地让某一方先行动完成后另一方再行动。结算时会综合考虑技能优先级、角色位置、目标格、技能范围、能量消耗、冷却时间和当前状态。这样的设计增强了战斗的策略性，玩家不能只根据当前敌人位置机械释放技能，而需要预判敌方移动和技能选择。

棋盘位置判定是战斗系统的关键组成部分。游戏中的技能并非全部自动命中，而是根据技能规则判断是否命中目标。例如普通攻击和重斩需要选择一定距离内的目标格，只有敌人在该格时才会受到伤害；贯穿射击要求敌人与释放者处于同一行或同一列，并且位于玩家选择的方向上；范围爆破、火焰龙卷、乱魂反噬等技能则根据区域覆盖范围判断敌人是否受到影响。通过目标格和范围判定，游戏引入了战棋类游戏中常见的走位与预判机制。

资源管理也是本项目战斗功能的重要内容。角色拥有生命值和能量值，技能通常需要消耗一定能量，并在使用后进入冷却。玩家不能无限制连续释放强力技能，而需要在攻击、防御、恢复和蓄能之间进行取舍。当技能能量不足时，系统会提示释放失败，并允许失败方重新选择行动，避免错误操作直接跳过整个回合。冷却系统则限制了高价值技能的连续使用，使玩家需要规划多个回合内的技能释放顺序。

状态系统为战斗增加了更多变化，同时也规定了相关技能的优先级。游戏中实现了闪避、格挡、盾墙、反击姿态、沉默、中毒、无敌、攻击削弱、移动削弱、随机移动、锁链牵制、血怒、不死、残骨复活等多种状态。不同状态会在不同阶段触发，有的在技能释放时生效，有的在受到伤害时生效，有的在回合开始或结束时更新。例如反击姿态可以在受到攻击时返还完整伤害，同时自身只承受部分伤害；翻滚可以在成功躲过敌方攻击后获得闪避状态；中毒会在后续回合持续造成无视防御的伤害；乱魂状态会让角色失去自主行动并随机移动。状态系统使战斗结果不仅取决于单次伤害，还取决于多个回合之间的效果积累。

程序的 AI 会根据角色状态、双方距离、技能可用性和当前生命值等因素选择行动。虽然 AI 不是复杂的机器学习模型，但它能够完成基本的战斗决策，例如在距离合适时使用攻击技能，在低生命值时优先使用保命技能，在技能可用时选择合适目标区域。AI 的存在使玩家无需双人操作即可完整体验游戏流程，也方便进行演示和测试。

为了提升对局过程的可读性，战斗界面提供了详细的信息展示。界面中会显示玩家和敌方角色的生命值、能量、当前位置、当前选择技能和目标格。战斗日志会记录每回合的技能释放、伤害结算、状态触发、移动结果、能量不足提示和胜负结果。玩家可以通过日志理解系统如何结算每一次行动，这对于回合制游戏尤为重要，也方便调试和展示程序逻辑。本项目还加入了可信对局展示机制。每回合 AI 的行动会先以加密承诺的形式显示在日志中，回合结算后再揭示明文行动。游戏结束时，系统会公布用于验证的 RSA 私钥、AES 密钥相关信息和各回合对局记录。该机制体现了“先承诺、后揭示”的思想，目的是证明 AI 不是在玩家选择之后临时改变行动，从而增强同步回合制对局的公平性和可信度。虽然该功能主要用于课程展示，但它体现了项目在游戏规则之外对公平性机制的进一步探索。

账号与战绩记录也是程序功能的一部分。用户登录后，程序会记录账号相关信息，包括

玩家头像、胜负统计和上一次选择的阵容配置。这样玩家再次进入游戏时，可以继续使用此前的角色和技能配置，而不必每次重新选择。战绩统计使程序具有一定持续使用价值，也让游戏流程更接近完整应用而不是单次运行脚本。

战斗结束后，程序会进入结算界面。根据战斗结果，系统会显示对应角色的胜利结算图，若双方同时倒下则显示平局图。每个角色都有独立的胜利图片和中文胜利短句，结算界面还提供“再来一局”和“返回主页”按钮。该设计使游戏从开始、战斗到结束形成闭环，也增强了视觉表现和完成度。

二、项目整体结构

1. 程序入口和页面总控：main.py 该部分是整个程序执行流程的控制中心
2. 角色基类和角色工厂：character.py, character_factory.py 存储着相关角色技能的接口、角色的数值、角色的生成、角色相关图片站位的接口，是整个游戏的核心部分。
3. 技能基类和技能工厂：skill.py, skill_factory.py 存储着技能相关的接口，数值，图片接口，是游戏的关键部分
4. 战斗状态与结算：battle_state.py 用于结算战斗状态，很多技能的释放是否合法，优先级的控制以及判断，都需要依靠这个模块
5. AI 决策控制：battle_ai.py 本部分是 ai 部署部分，ai 的三个难度选择都在此模块完成
6. 登录与用户数据：auth_service.py, login_page.py 该部分负责构造登陆界面和用户数据的保存和加载
7. 可信对局加密：crypto.py 这部分将 ai 的决策加密吐出来，然后反馈到对局 ui 上
8. Ui 组件：image_page.py, skill_button.py 这部分负责构造 ui 接口与相关图片

项目核心模板可以概括为三层：

数据模板层：定义角色、技能、配置表。

逻辑模板层：由 BattleState 统一处理技能释放、移动、伤害、状态、冷却、能量等战斗规则。

表现模板层：由 main.py 中的页面类和 UI 组件负责界面显示、按钮交互和页面跳转。

角色与技能都不是直接固定在战斗流程中，而是通过“基类 + 子类 + 工厂注册”的方式接入。这样新增角色或技能时，只需要新建子类并在工厂中登记，不需要大规模修改战斗主循环。

三、类模板介绍

1. 角色基类 character:

角色基类的变量主要有：

- name: 角色名称
- max_hp、hp: 最大生命和当前生命
- pos: 棋盘位置
- shield: 护盾值（已删除，保留仅作兼容）
- attack: 基础攻击力
- max_energy、energy: 能量上限和当前能量
- move_power: 移动能力
- exclusive_skill_ids: 角色专属技能 ID
- basic_skill_ids: 通用技能 ID
- sprite_path、sprite_size: 角色图片资源

- status、special_state: 普通状态和特殊状态字典

关键方法:

on_round_end(): 预留回合结束钩子。

all_skill_ids(): 返回专属技能 + 通用技能。

get_sprite_path(): 返回角色图片路径。

get_sprite_size(): 返回角色显示尺寸。

现有角色子类都继承 Character, 主要在 __init__ 中设置角色基础数值和专属技能。例如:

FireMage:

EarthMage:

VineMage:

SpaceMage:

WarLord:

ChainGladiator:

ShadowAssassin:

BloodBerserker:

Birdman:

LizardRogue:

BoneGeneral:

WitchDoctor:

角色工厂 create_character 根据 character_id 创建对应角色对象, 是角色模板的统一入口。

2. 技能基类 skill:

技能基类是 Skill

Skill 保存所有技能通用属性:

skill_id: 技能唯一 ID

name: 技能名称

description: 技能描述

cooldown: 冷却回合

current_cd: 当前剩余冷却

target_required: 是否需要目标格

range: 技能范围

energy_cost: 能量消耗

关键方法:

can_use(): 判断冷却和能量是否允许释放。

start_cooldown(): 进入冷却。

reduce_cooldown(): 回合结束减少冷却。

cast(): 技能释放接口, 子类必须重写。

on_turn_start() / on_turn_end(): 持续效果钩子。

3. 战斗结算核心类 effect 和 battlestate:

战斗核心由 Effect 和 BattleState 组成

Effect 是一个延迟结算对象, 包含:

priority: 优先级

actor: 行动者

resolve_fn: 实际执行函数

data: 效果参数

BattleState 是战斗状态总管理类, 它管理棋盘大小、双方角色、技能表、当前选择、效果队列、泥潭格、平局状态等。

核心方法:

push(): 向效果队列加入一个效果。

push_damage(): 加入伤害效果。

push_status(): 加入状态效果。

push_move(): 加入移动效果。

resolve(): 一回合的总结算入口。

resolve() 的流程是本项目战斗模板的核心:

清空上一轮效果队列。

检查双方技能能量、冷却、混乱状态。

调用双方技能的 cast(), 但不立即结算。

将技能产生的伤害、移动、状态等效果压入队列。

按优先级排序并执行效果。

处理死亡免疫、平局、闪避奖励、状态清理等回合末逻辑。

4. 主程序与页面类:

主程序入口是 main(), 最终通过 if __name__ == "__main__" 启动,

main.py 中还集中保存了角色和技能展示配置:

HERO_CONFIGS: 角色选择界面的角色配置表。

get_hero_config() 按角色 ID 查配置。

DEFAULT_BASIC_SKILL_IDS: 默认通用技能。

GENERIC_SKILL_CONFIGS: 通用技能选择界面配置。

get_skill_detail_info(): 读取技能详情用于 UI 展示。

主要页面类:

HomePage: 主页。

CardSelectPage: 卡牌/流程选择页。

SkillSelectPage: 通用技能选择页。

HeroSelectPage: 角色选择页。

AISelectPage: AI 难度选择页。

ResultPage: 结算页。

BattleCanvas: 战斗棋盘绘制类。

BattlePage: 战斗页面核心类。

MainWindow: 窗口和页面栈总控。

其中 BattlePage.__init__() 负责创建玩家、敌人、技能列表、BattleState、AI 控制器和加密引擎。

confirm_action() 是玩家确认行动后的主要入口。调用战斗状态完成一回合结算。

5. Ai 类:

AI 配置通过 AI_CONFIGS 定义不同难度的角色、技能和说明。

BattleAIController 总决策入口。

_choose_easy(): 简单 AI, 偏随机和权重选择。

_choose_normal(): 普通 AI, 根据距离、血量、攻击范围做选择。

_choose_hard(): 困难 AI, 使用局部搜索和收益评估。

AI 输出格式统一为 (actor, skill, target), 和玩家行动格式一致, 因此可以直接交给 BattleState.resolve() 统一结算。

6. 加密类:

GameCryptoEngine 位于用于实现 AI 预提交机制。

核心方法:

start_new_game(): 生成 RSA 密钥对和 AES 会话密钥, 并公开 RSA 公钥及加密后的 AES 密钥。

commit_action(): 把 AI 本回合选择加密成密文, 先展示给玩家。

reveal_private_key(): 游戏结束公开私钥, 供玩家验证 AI 是否提前承诺行动。

这部分与 BattlePage 的 AI 出招逻辑结合, 主要调用点在:

初始化加密

AI 预提交

当前回合揭示

私钥公开

四、组员分工

陈勇整: 游戏设计、数据调整、游戏主体函数、游戏对局界面制作、游戏主页制作、游戏角色选择界面制作、第一行 basic 技能制作, 法师类角色制作, bug 修改

袁方涵: 数据调整、游戏选 ai 界面制作, 技能选择界面制作, ai 相关代码制作, 第二行 basic 技能制作, 兽人类角色制作, bug 修改

胡翔宇: 数据调整, 游戏登录界面制作, 信息存储制作, 战斗逻辑修改制作, 第三行 basic 技能制作, 战士类角色制作, bug 修改

五、项目总结和反思

本项目是一款回合制战术对战游戏, 实现了一个包含角色选择、通用技能配置、AI 对战、战斗结算、胜负展示和账户记录等功能的完整桌面游戏程序。游戏以网格地图为核心战斗空间, 玩家需要在每一回合中综合考虑角色位置、技能范围、能量消耗、冷却时间、命中判定和敌方可能行动, 从而完成攻击、防御、位移、控制和资源管理等决策。相比单纯的数值对抗, 本项目更强调策略性和可操作性, 使玩家能够通过技能组合和位置判断影响战斗结果。

在开发过程中, 我们将程序拆分为界面交互、角色系统、技能系统、战斗结算、AI 决策、登录统计和资源管理等模块。这样的结构使项目逻辑更加清晰, 也便于后期维护和扩展。例如, 角色类主要负责保存生命值、攻击力、能量、移动力和专属技能等数据; 技能类负责描述技能名称、效果、冷却、能耗、目标选择方式和释放逻辑; 战斗状态类统一处理双方行动、伤害结算、状态变化、冷却更新和胜负判断。通过这种模块化设计, 新角色和新技能可以在相对独立的文件中添加, 不需要大范围修改主流程代码。

本项目的重点难点是技能效果与战斗结算逻辑的统一。游戏中既有普通攻击、治疗、格挡、位移等通用技能, 也有各角色的专属技能, 不同技能在目标选择、作用范围、命中条件和后续状态上存在较大差异。开发中我们逐步发现, 如果只在界面层限制目标选择, 很容易造成“看起来可以释放, 但实际逻辑不一致”的问题。因此后期将技能范围、目标格选择、

命中判定和冷却消耗进一步统一到战斗逻辑中，使技能即使未命中也能按照设计进入冷却，避免出现规则漏洞。

另一个重要收获是对用户界面信息展示的重视。最初技能选择界面只关注玩家能否选择技能，但对于技能说明、能量消耗和冷却时间展示不足，导致玩家很难做出合理判断。后续我们补充了技能介绍、CD、能耗等信息，并为角色信息、胜利结算图和报告展示素材进行了整理。这说明游戏项目不仅需要底层逻辑正确，也需要把规则清晰地呈现给玩家，否则复杂系统会降低可玩性。

在调试过程中，我们也暴露出一些不足。首先，技能数量较多，人工测试容易遗漏边界情况，例如能量不足时是否重新选择、范围技能是否正确判定目标、未命中是否进入冷却等。后续如果继续完善，应当为核心战斗逻辑编写更多自动化测试，用固定位置、固定技能和固定回合验证各种结算结果。其次，部分技能说明和实际效果曾经存在不一致，这说明数据配置、界面显示和战斗实现之间还需要建立更严格的同步机制。比较理想的做法是让技能说明、能耗、冷却和效果参数尽量来自统一的数据结构，减少手动维护带来的错误。

总体来看，本项目完成了从游戏规则设计到可运行程序实现的完整流程。通过这次开发，我们对面向对象设计、事件驱动界面、状态管理、复杂条件判断和资源组织都有了更深入的理解。项目也让我们认识到，程序设计不仅是把功能写出来，还包括如何组织代码、如何降低模块之间的耦合、如何让用户理解系统规则，以及如何通过测试保证复杂逻辑的稳定性。后续如果继续迭代，可以进一步完善 AI 策略、增加更多地图机制、优化动画表现，并将角色和技能配置改为外部数据文件，使游戏具有更强的扩展性和可维护性。